Introduction to WebSockets with Socket.IO

Presented by:

Cristian Camilo Cortes Ortiz

Codecamp academy

Table of contents

Audience 3
Purpose 3
Introduction 3
What are WebSockets? 4
- HTTP versus WebSockets 4
- Techniques Before WebSockets 6
- The WebSocket Handshake 7
- The Browser WebSocket API 8

Main Concepts 9
- Full-Duplex 9
- Persistent Connection 9
- Handshake 9
- Frames 9
- Events 9
- Broadcasting 9
- Rooms 10
- Namespaces 10
- Acknowledgements 10
- Transports 10

What is Socket.IO? 11
- Main Features 11
- Architecture 11
- WebSocket versus Socket.IO 12

Setting Up the Project 12
- Requirements 12
- Installing the Dependencies 13
- Project Configuration 13

Creating the Server 14
- Attaching Socket.IO to an HTTP Server 14
- Server Options 15
- Running the Server 15

Creating the Client 16
- Loading the Client Library 17
- Connecting to Other Servers 17

Events 17
- Built-in Events 17

Custom Events 17
Naming Convention 18
Sending Data to Clients 19
Acknowledgements 21
Rooms 22
Namespaces 23
Middleware and Authentication 24
Disconnections and Reconnection 25
Disconnect Reasons 25
Automatic Reconnection 26
Connection State Recovery 27
Practical Example: Chat Application 28
Server 29
Client 30
Scaling Socket.IO 31
Practice Exercises 31
Best Practices 32
Conclusion 32

Audience

This manual contains technical terminology and is intended for software development students, instructors, and developers who want to learn the fundamentals of real-time communication with WebSockets using the Socket.IO library in Node.js. It covers the basic concepts needed to build bidirectional, event-driven applications, from the WebSocket protocol to events, rooms, namespaces, and authentication. Readers should be familiar with JavaScript, asynchronous programming, and the basics of Node.js and Express.

Purpose

The purpose of this manual is to give readers a clear understanding of how real-time, bidirectional communication works between a client and a server, and how to implement it in Node.js with Socket.IO. By the end of this document, the reader will be able to:

- Explain the difference between the HTTP request-response model and WebSockets.
- Describe how the WebSocket handshake and the persistent connection work.
- Explain what Socket.IO adds on top of the WebSocket protocol.
- Create a Socket.IO server and client in Node.js.
- Send and receive events, use acknowledgements, and broadcast messages.
- Organize clients with rooms and namespaces.
- Protect connections with middleware and authentication.
- Handle disconnections and reconnections, and apply good practices.

Introduction

Traditional web applications follow a request-response model: the client always asks and the server answers. This model works well for loading pages and consuming REST APIs, but it falls short when the server must notify the client as soon as something happens, as in chats, live dashboards, notifications, online games, or collaborative editing. WebSockets remove this limitation by keeping a single connection open through which both sides can send messages at any time.

Socket.IO is the most popular library to work with this technology in Node.js, because it adds automatic reconnection, rooms, acknowledgements, and a simple event-based API on top of it. This document starts with the theory of WebSockets, presents Socket.IO and its architecture, and then implements each fundamental concept step by step until building a complete chat application.

What are WebSockets?

WebSocket is a communication protocol, standardized in RFC 6455, that provides a full-duplex channel over a single TCP connection. It starts as a regular HTTP request and is then upgraded, so it works through the same ports used by the web: 80 for unencrypted connections and 443 for connections protected with TLS.

HTTP versus WebSockets

In the HTTP model, the client always starts the conversation: the server can only answer a request; it cannot send data on its own. With WebSockets, after an initial handshake, the connection remains open, and either side can send messages whenever it needs to. Figure 1 compares both models.

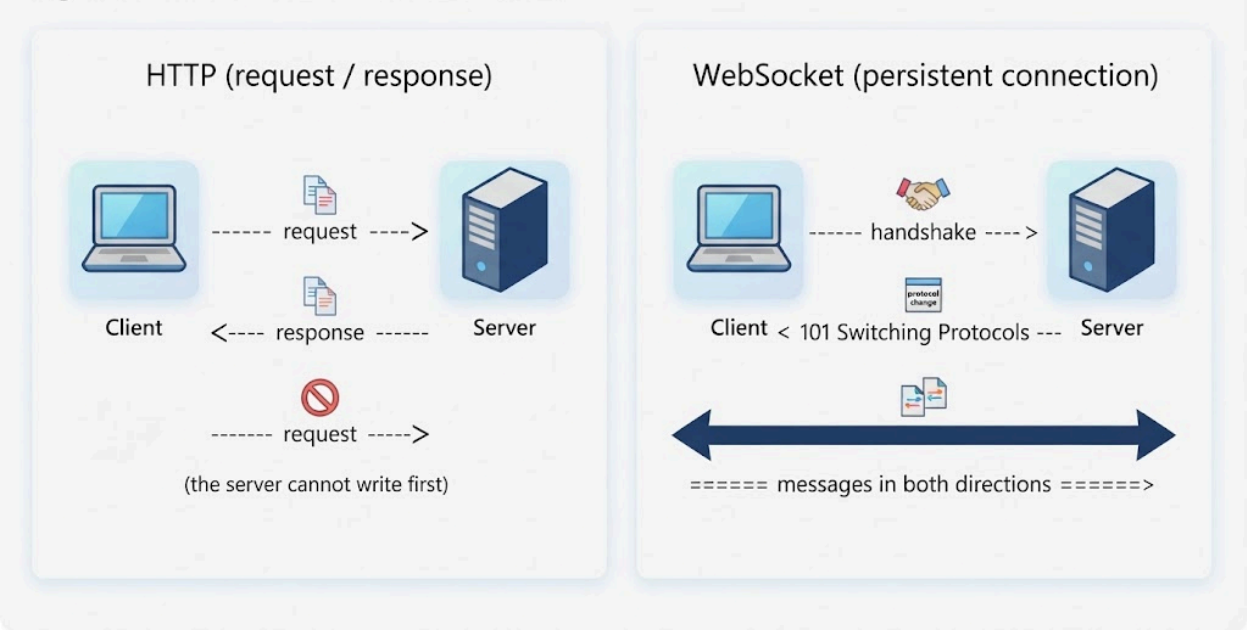


Figure 1. Message flow in HTTP and in WebSocket

Table 1 summarizes the main differences between both protocols.

Aspect	HTTP	WebSocket
Communication	Client request, server response	Messages in both directions at any time
Connection	Opened per request or reused with keep-alive	One persistent connection
Overhead	Headers sent with every request	Small frame header after the handshake
URL scheme	http:// and https://	ws:// and wss://
Typical use	Pages and REST APIs	Chats, live dashboards, games, notifications

Table 1. HTTP versus WebSocket

Techniques Before WebSockets

Before WebSockets existed, developers simulated real-time communication using only HTTP. These techniques still appear in legacy systems, and Socket.IO uses one of them, long polling, as a fallback. Table 2 compares them.

Technique	How it works	Direction	Drawback
Polling	The client repeats requests at fixed intervals	Client to server	Wasted requests and delayed updates
Long polling	The server holds the request open until it has data	Server to client	A new request is needed after every message
Server-Sent Events	The server pushes text events over one HTTP response	Server to client	One-way channel only
WebSocket	A persistent full-duplex connection	Both directions	Stateful servers and reconnection logic

Table 2. Real-time communication techniques

The WebSocket Handshake

A WebSocket connection begins as a normal HTTP GET request that includes the Upgrade: websocket header. If the server accepts, it answers with the status code 101 (Switching Protocols) and a Sec-WebSocket-Accept header calculated from the key sent by the client. From that moment, the same TCP connection stops speaking HTTP and starts exchanging WebSocket frames. Figure 2 shows both messages. Browsers use ws:// for unencrypted connections and wss:// for connections protected with TLS.

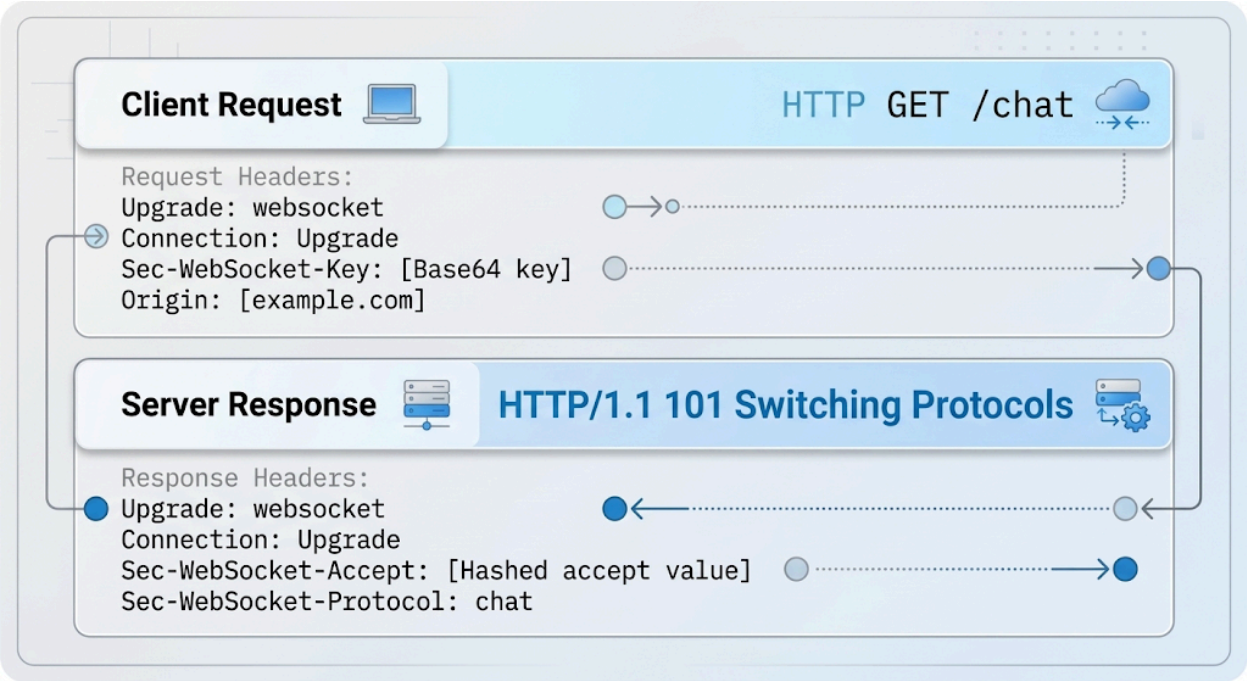


Figure 2. Opening handshake of a WebSocket connection

The Browser WebSocket API

Browsers include a native WebSocket object, shown in Figure 3. It is enough for simple cases, but it is a low-level API: it only sends strings or binary data, it does not reconnect by itself, and it has no concept of event names, rooms, or acknowledgements. These are the gaps that Socket.IO fills.



```
const ws = new WebSocket('ws://localhost:8080');

ws.onopen = () => ws.send('Hello server');
ws.onmessage = (event) => console.log(event.data);
ws.onclose = () => console.log('Connection closed');
ws.onerror = (error) => console.error(error);

```

Figure 3. Native WebSocket API in the browser

Main Concepts

The following concepts are the foundation for understanding real-time communication with WebSockets and Socket.IO.

Full-Duplex

Full-duplex means that the client and the server can send data at the same time, without waiting for each other.

Persistent Connection

After the handshake, the connection stays open until one of the sides closes it. This removes the cost of opening a new connection for every message.

Handshake

The handshake is the initial HTTP exchange that upgrades a connection to the WebSocket protocol.

Frames

Frames are the small units in which WebSocket data travels. There are data frames, which carry text or binary messages, and control frames, such as ping, pong, and close.

Events

In Socket.IO, every message is an event identified by a name and accompanied by data of any serializable type. One side emits the event, and the other side listens for it.

Broadcasting

Broadcasting is sending an event to many clients at once, for example, to every user connected to a chat.

Rooms

A room is a server-side group of sockets that makes it possible to send events only to some of the connected clients.

Namespaces

A namespace is a communication channel identified by a path, such as /admin, that separates the logic of an application while sharing a single underlying connection.

Acknowledgements

An acknowledgement is a callback that the receiver of an event invokes to confirm that it processed the event, optionally returning data to the sender.

Transports

A transport is the mechanism used to carry data between the client and the server. Socket.IO supports HTTP long-polling and WebSocket.

What is Socket.IO?

Socket.IO is a library that enables real-time, bidirectional, and event-based communication between a client and a server. It provides a server for Node.js and client libraries for the browser, Node.js, and other languages. It is important to clarify that Socket.IO is not an implementation of the WebSocket protocol: it uses WebSocket as one of its transports, but it adds extra information to every packet. For this reason, a plain WebSocket client cannot connect to a Socket.IO server, and a Socket.IO client cannot connect to a plain WebSocket server.

Main Features

- Event-based API with named events and any serializable data.
- Automatic reconnection with an increasing delay between attempts.
- Fallback to HTTP long-polling when WebSocket is not available.
- Acknowledgements, broadcasting, rooms, and namespaces.
- Support for binary data.
- Connection state recovery after temporary disconnections (since version 4.6).

Architecture

Socket.IO is built in two layers. The lower layer, Engine.IO, establishes the connection, selects the transport, and sends the heartbeat. The upper layer, Socket.IO itself, adds events, namespaces, rooms, and acknowledgements.

By default, a client first connects using HTTP long-polling, which works in almost any network, and then upgrades to WebSocket when it is available. Once connected, the server sends a ping packet every pingInterval milliseconds (25 seconds by default), and the client must answer with a pong packet within pingTimeout milliseconds (20 seconds by default). Otherwise, the connection is considered closed.

WebSocket versus Socket.IO

Table 3 compares the native WebSocket API with the features offered by Socket.IO.

Feature	Native WebSocket	Socket.IO
Reconnection	Manual	Automatic
Message identification	A single stream of messages	Named events
Rooms and namespaces	Not available	Built in
Acknowledgements	Not available	Built in
Fallback transport	None	HTTP long-polling
Heartbeat	Implemented by the application	Built in (ping and pong)

Table 3. Native WebSocket versus Socket.IO

Setting Up the Project

Requirements

To follow the examples in this manual, the following tools are needed:

- Node.js (a current LTS version) and npm.
- Basic knowledge of Express.
- A browser and a code editor.

All the examples use Socket.IO version 4 and ES modules.

Installing the Dependencies

Figure 4 creates the project and installs Express, which serves the HTML page, and Socket.IO.



Figure 4. Creating the project and installing the dependencies

Project Configuration

Add the type property with the value module to the package.json file so that Node.js accepts the import syntax, and define the scripts shown in Figure 5. The project uses two files: server.js, the entry point with the Express and Socket.IO server, and public/index.html, the client page served as a static file.

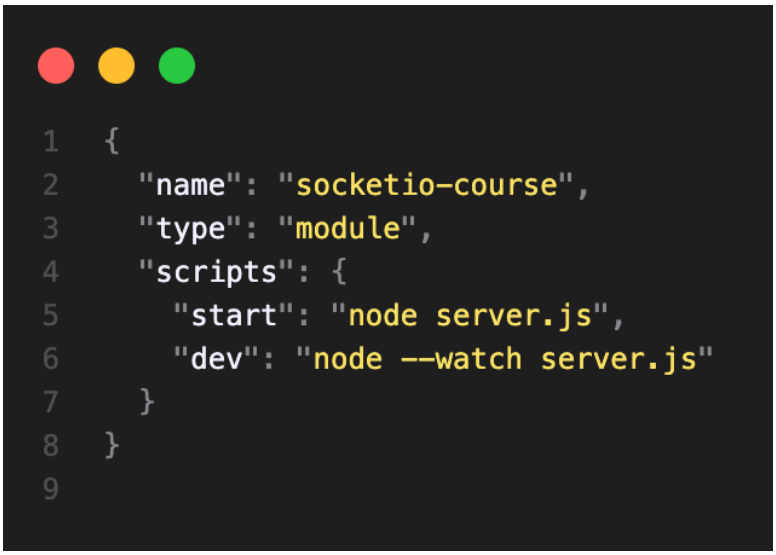


Figure 5. Relevant properties of package.json

Creating the Server

Attaching Socket.IO to an HTTP Server

Socket.IO does not replace Express: it attaches to the same Node.js HTTP server. Express handles the regular HTTP routes and static files, while Socket.IO handles the real-time connections. Figure 6 shows a minimal server.

```
import express from 'express';
import { createServer } from 'node:http';
import { Server } from 'socket.io';

const app = express();
const httpServer = createServer(app);
const io = new Server(httpServer);

app.use(express.static('public'));

io.on('connection', (socket) => {
  console.log('Client connected:', socket.id);

  socket.on('disconnect', (reason) => {
    console.log('Client disconnected:', socket.id, reason);
  });
});

httpServer.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

Figure 6. Minimal Express and Socket.IO server (server.js)

The io object is the Socket.IO server. Each time a client connects, the connection event runs and receives a socket, the object that represents that specific client and has a unique ID. All the listeners for that client are registered inside this callback, including the disconnect event, which receives the reason for the disconnection.

Server Options

The Server constructor accepts an optional object with configuration options. Table 4 lists the most common ones. For example, new Server(httpServer, { cors: { origin: 'http://localhost:5173' } }) allows a front-end served from another origin to connect.

Option	Description	Default
cors	Origins and methods allowed to connect from other domains	Same origin only
transports	Enabled transports	polling, websocket
pingInterval	Milliseconds between heartbeat pings	25000
pingTimeout	Milliseconds to wait for the pong before closing	20000
maxHttpBufferSize	Maximum size of a message in bytes	1e6 (1 MB)
connectionStateRecovery	Restores the state of clients after a temporary disconnection	Disabled

Table 4. Common options of the Socket.IO server

Running the Server

Run the npm run dev command from the project folder. Node.js restarts the server automatically every time a file changes. Once the client in the next section is ready, open http://localhost:3000 in two browser tabs: the terminal prints one line with a different socket ID for each tab, which confirms that both clients are connected.

Creating the Client

The client is an HTML page that loads the Socket.IO client library and calls the io function to open the connection. Save the code of Figure 7 as public/index.html.

```html
<!DOCTYPE html>
<html lang="en">
<body>
  <h1>Socket.IO client</h1>

  <script src="/socket.io/socket.io.js"></script>
  <script>
    const socket = io();

    socket.on('connect', () => {
      console.log('Connected with id', socket.id);
    });

    socket.on('disconnect', (reason) => {
      console.log('Disconnected:', reason);
    });
  </script>
</body>
</html>

```

Figure 7. Minimal Socket.IO client (public/index.html)

Loading the Client Library

The server serves the client bundle automatically at /socket.io/socket.io.js, which defines the global function io. Calling io without arguments connects to the same host that served the page.

Connecting to Other Servers

When the front-end and the server live in different origins, pass the URL to the function, as in io('https://api.example.com'), and configure the CORS option on the server. In projects that use a bundler, such as Vite, or in Node.js clients, install the socket.io-client package with npm and import the function with import { io } from 'socket.io-client'.

Events

Communication in Socket.IO is based on events: one side emits an event with a name and some data, and the other side listens for it using the same name.

Built-in Events

Socket.IO emits some events by itself to report the state of the connection:

- connection: on the server, runs when a client connects.
- connect: on the client, runs when the connection is established, including after a reconnection.
- disconnect: on both sides, runs when the connection is lost and receives the reason.
- connect_error: on the client, runs when the connection fails or is rejected by a middleware.

Custom Events

Applications define their own events with the emit and on methods. In Figure 8, the client sends a chat:message event and the server forwards it to every connected client. Data can be strings, numbers, booleans, objects, arrays, or binary data such as a Buffer. Objects are serialized to JSON automatically, so functions and circular references cannot be sent. The names connect, connect_error, disconnect, disconnecting, newListener, and removeListener are reserved and cannot be used as custom events.

```
// Client
socket.emit('chat:message', { user: 'Maria', text: 'Hello everyone' });

socket.on('chat:message', (data) => {
  console.log(`${data.user}: ${data.text}`);
});

// Server
io.on('connection', (socket) => {
  socket.on('chat:message', (data) => {
    io.emit('chat:message', data);
  });
});
```

Figure 8. Emitting and listening for a custom event

Naming Convention

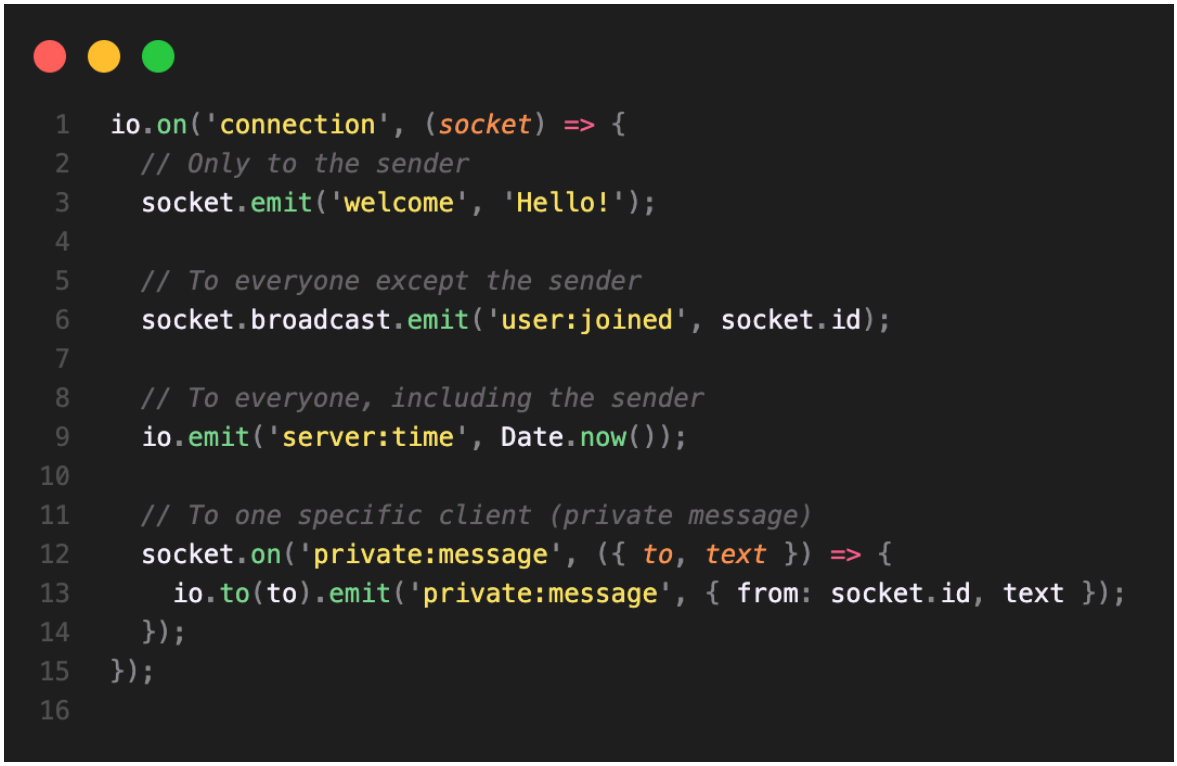
A common convention is to name events with the pattern domain:action, such as chat:message, room:join, or user:typing. It keeps the events organized as the application grows and avoids name collisions.

Sending Data to Clients

Besides answering the client that sent an event, the server can send events to many clients. The method used determines who receives the event, as summarized in Table 5 and shown in Figure 9.

Method	Receivers
socket.emit()	Only the client of that socket
socket.broadcast.emit()	All connected clients except the sender
io.emit()	All connected clients
io.to(room).emit()	All clients in the room
socket.to(room).emit()	All clients in the room except the sender
io.except(room).emit()	All clients except those in the room
io.to(socketId).emit()	One specific client (private message)

Table 5. Methods to send events from the server



```
io.on('connection', (socket) => {
  // Only to the sender
  socket.emit('welcome', 'Hello!');

  // To everyone except the sender
  socket.broadcast.emit('user:joined', socket.id);

  // To everyone, including the sender
  io.emit('server:time', Date.now());

  // To one specific client (private message)
  socket.on('private:message', ({ to, text }) => {
    io.to(to).emit('private:message', { from: socket.id, text });
  });
});
```

Figure 9. Sending events to different receivers

Acknowledgements

By default, emit is fire-and-forget: the sender does not know whether the other side received the event. An acknowledgement is a callback passed as the last argument of emit; the receiver invokes it to confirm the event and, optionally, to return data, as in Figure 10. It is the Socket.IO equivalent of a request and its response. Adding timeout() makes the callback receive an error if no answer arrives in the given time, and since version 4.6 the emitWithAck method returns a promise that can be used with async/await.

```
// Client: callback version
socket.timeout(5000).emit('order:create', { item: 'book' }, (error, response) => {
  if (error) return console.error('The server did not answer in time');
  console.log('Order id:', response.id);
});

// Client: promise version, inside an async function
const response = await socket.timeout(5000).emitWithAck('order:create', { item: 'book' });

// Server
socket.on('order:create', (order, callback) => {
  const id = Date.now();
  callback({ status: 'ok', id });
});

```

Figure 10. Acknowledgement of an event

Rooms

A room is a server-side channel that sockets can join and leave. Rooms make it possible to send events to a subset of clients, such as the members of a chat group or the viewers of a specific document. Figure 11 implements joining, leaving, and sending messages to a room.

```
io.on('connection', (socket) => {
  socket.on('room:join', (room) => {
    socket.join(room);
    socket.to(room).emit('room:notice', `${socket.id} joined ${room}`);
  });

  socket.on('room:leave', (room) => {
    socket.leave(room);
  });

  socket.on('room:message', ({ room, text }) => {
    if (!socket.rooms.has(room)) return;
    io.to(room).emit('room:message', { from: socket.id, text });
  });
});

```

Figure 11. Joining, leaving, and sending messages to a room

Keep in mind the following rules about rooms:

- Rooms exist only on the server; clients do not know which rooms they belong to.
- Every socket joins automatically a room named with its own ID, which is why io.to(socketId) works.
- When a socket disconnects, it leaves all its rooms.
- Use socket.rooms to check that a client belongs to a room before sending messages on its behalf.

Namespaces

A namespace is a communication channel identified by a path that splits the logic of an application over a single shared connection. The default namespace is /. Each namespace has its own events, rooms, and middleware, as shown in Figure 12. Table 6 compares rooms and namespaces.

```
// Server
const adminNamespace = io.of('/admin');

adminNamespace.on('connection', (socket) => {
  console.log('Admin connected:', socket.id);
  socket.emit('stats', { users: io.engine.clientsCount });
});

// Client
const adminSocket = io('/admin');
adminSocket.on('stats', (stats) => console.log(stats));

```

Figure 12. Creating and connecting to a namespace

Aspect	Rooms	Namespaces
Defined by	The server, with join and leave	The server, with io.of()
Chosen by the client	No	Yes, by connecting to its path
Purpose	Group sockets dynamically	Separate application concerns
Middleware	Shared with the namespace	Its own middleware
Lifetime	Created and removed on demand	Usually fixed at startup

Table 6. Rooms versus namespaces

Middleware and Authentication

Middleware functions run once for each incoming connection, before the connection event. They are the right place to authenticate users, as in Figure 13. The client sends its credentials in the auth option, the server reads them from socket.handshake.auth, and calling next with an error rejects the connection, which makes the client emit connect_error. The socket.data property stores information about the connected user, such as its identifier or role.

```
// Server
io.use((socket, next) => {
  const token = socket.handshake.auth.token;

  if (token !== process.env.SOCKET_TOKEN) {
    return next(new Error('Unauthorized'));
  }

  socket.data.userId = 'user-123';
  next();
});

// Client
const socket = io({ auth: { token: 'my-secret-token' } });

socket.on('connect_error', (error) => {
  console.error('Connection refused:', error.message);
});

```

Figure 13. Authentication middleware

A fixed token is used here only for teaching purposes. In a real project, the client should send a verified token, such as a JWT, or rely on a session. Socket.IO also offers per-packet middleware with socket.use(), which runs for every event received from a client.

Disconnections and Reconnection

Connections can be lost at any time due to network failures, server restarts, or user actions. A real-time application must expect this and recover from it.

Disconnect Reasons

Socket.IO identifies the cause of a disconnection through the reason received by the disconnect event, summarized in Table 7.

Reason	Meaning	Automatic reconnection
io server disconnect	The server forced the disconnection with socket.disconnect()	No
io client disconnect	The client called socket.disconnect()	No
ping timeout	The other side did not answer the heartbeat in time	Yes
transport close	The connection was closed, for example by a network loss	Yes
transport error	The connection found an error	Yes

Table 7. Reasons for the disconnect event

Automatic Reconnection

By default, the client tries to reconnect automatically, increasing the delay between attempts. This behavior can be tuned with options and observed through the events of the socket.io manager object, as shown in Figure 14. When the server itself closes the connection (io server disconnect), the client does not reconnect and must call socket.connect() manually.

```
const socket = io({
  reconnectionAttempts: 5,
  reconnectionDelay: 1000,
  reconnectionDelayMax: 5000
});

socket.on('disconnect', (reason) => {
  if (reason === 'io server disconnect') {
    socket.connect(); // manual reconnection
  }
});

socket.io.on('reconnect_attempt', (attempt) => {
  console.log('Reconnection attempt', attempt);
});

socket.io.on('reconnect', () => {
  console.log('Reconnected');
});

```

Figure 14. Configuring and observing reconnection in the client

Connection State Recovery

After a reconnection, the socket gets a new ID, loses its rooms, and misses the events emitted during the interruption. Connection state recovery, available since Socket.IO 4.6, temporarily stores the state of the disconnected client so that it can be restored if the client returns in time, as shown in Figure 15. When socket.recovered is true, the id, the rooms, and the missed events are restored. Otherwise, the application must synchronize the state by itself, for example by requesting the latest messages from the database.



```
const io = new Server(httpServer, {
  connectionStateRecovery: {
    maxDisconnectionDuration: 2 * 60 * 1000,
    skipMiddlewares: true
  }
});

io.on('connection', (socket) => {
  if (socket.recovered) {
    console.log('Session recovered:', socket.id);
  } else {
    console.log('New session:', socket.id);
  }
});

```

Figure 15. Enabling connection state recovery

Practical Example: Chat Application

To integrate the concepts of this manual, the following example builds a chat application with a username, a shared room, a typing indicator, and acknowledgements. It combines the following concepts:

- Middleware: the username is received and validated before the connection is accepted.
- Rooms: every client joins the general room.
- Events and broadcasting: messages, notices, and typing indicators reach the other users of the room.
- Acknowledgements: the server confirms or rejects every message.

Server

Figure 16 shows the complete server, which replaces the content of server.js. The server validates the type and the length of every message before sending it to the room.

```
import express from 'express';
import { createServer } from 'node:http';
import { Server } from 'socket.io';

const app = express();
const httpServer = createServer(app);
const io = new Server(httpServer);

app.use(express.static('public'));

io.use((socket, next) => {
  const username = socket.handshake.auth.username?.trim();
  if (!username) return next(new Error('Username is required'));
  socket.data.username = username;
  next();
});

io.on('connection', (socket) => {
  const { username } = socket.data;
  socket.join('general');
  socket.to('general').emit('chat:notice', `${username} joined the chat`);

  socket.on('chat:message', (text, callback) => {
    if (typeof text !== 'string' || !text.trim()) {
      return callback?.({ ok: false, error: 'Empty message' });
    }

    io.to('general').emit('chat:message', {
      user: username,
      text: text.trim().slice(0, 500)
    });
    callback?.({ ok: true });
  });

  socket.on('chat:typing', () => {
    socket.to('general').emit('chat:typing', username);
  });

  socket.on('disconnect', () => {
    socket.to('general').emit('chat:notice', `${username} left the chat`);
  });
});

httpServer.listen(3000, () => {
  console.log('Chat running on http://localhost:3000');
});

```

Figure 16. Complete chat server (server.js)

Client

Figure 17 shows the client, which replaces the content of public/index.html. Received text is rendered with textContent instead of innerHTML, so that a malicious message cannot inject HTML into the page. To test the application, run npm run dev, open http://localhost:3000 in two tabs, choose different names, and exchange messages.

```html
<!DOCTYPE html>
<html lang="en">
<body>
  <ul id="messages"></ul>
  <p id="typing"></p>
  <form id="form">
    <input id="input" autocomplete="off" placeholder="Type a message" />
    <button>Send</button>
  </form>

  <script src="/socket.io/socket.io.js"></script>
  <script>
    const username = prompt('Your name:') || 'Guest';
    const socket = io({ auth: { username } });

    const messages = document.getElementById('messages');
    const typing = document.getElementById('typing');
    const form = document.getElementById('form');
    const input = document.getElementById('input');
    let typingTimer;

    const addLine = (text) => {
      const li = document.createElement('li');
      li.textContent = text;
      messages.appendChild(li);
    };

    form.addEventListener('submit', (event) => {
      event.preventDefault();
      if (!input.value) return;
      socket.emit('chat:message', input.value, (response) => {
        if (!response.ok) addLine(`Error: ${response.error}`);
      });
      input.value = '';
    });

    input.addEventListener('input', () => socket.emit('chat:typing'));

    socket.on('chat:message', ({ user, text }) => addLine(`${user}: ${text}`));
    socket.on('chat:notice', (text) => addLine(`* ${text}`));
    socket.on('chat:typing', (user) => {
      typing.textContent = `${user} is typing...`;
      clearTimeout(typingTimer);
      typingTimer = setTimeout(() => (typing.textContent = ''), 1000);
    });
    socket.on('connect_error', (error) => addLine(`Connection error: ${error.message}`));
  </script>
</body>
</html>
```

Figure 17. Complete chat client (public/index.html)

Scaling Socket.IO

Each Socket.IO server instance only knows its own clients. When an application runs several instances behind a load balancer, two things are needed. First, sticky sessions, so that all the HTTP long-polling requests of a client reach the same instance; they are not needed if only the WebSocket transport is enabled. Second, an adapter that forwards events between instances, so that a broadcast reaches clients connected to any of them. Figure 18 configures the Redis adapter, which requires installing the @socket.io/redis-adapter and redis packages.

```
import { createAdapter } from '@socket.io/redis-adapter';
import { createClient } from 'redis';

const pubClient = createClient({ url: 'redis://localhost:6379' });
const subClient = pubClient.duplicate();

await Promise.all([pubClient.connect(), subClient.connect()]);

io.adapter(createAdapter(pubClient, subClient));

```

Figure 18. Configuring the Redis adapter

Practice Exercises

The following exercises extend the chat application and reinforce the concepts of this manual:

- Show the list of connected users and update it on every connection and disconnection.
- Create several chat rooms with buttons to join and leave them.
- Implement private messages between two users with io.to(socketId).
- Build an /admin namespace protected with its own middleware.
- Use acknowledgements with a timeout to show a sent indicator next to every message.
- Enable connection state recovery and test it by stopping the network for a few seconds.

Best Practices

The following recommendations help keep real-time applications secure, reliable, and easy to maintain:

- Use wss:// and HTTPS in production.
- Configure CORS with explicit origins instead of allowing every origin.
- Authenticate in middleware and never trust data sent by the client: validate the type and the length of every payload.
- Render received text with textContent, not innerHTML, to avoid XSS attacks.
- Name events with the domain:action pattern.
- Use acknowledgements with timeouts for important operations.
- Register the listeners of a socket once, inside the connection event, and never inside other listeners, to avoid duplicated handlers and memory leaks.
- Limit the message size with maxHttpBufferSize and apply rate limiting to avoid abuse.
- Store important data in a database, not only in the memory of the process, and use an adapter when running several instances.
- Handle connect_error and disconnect in the client to keep the user informed.

Conclusion

WebSockets allow a client and a server to exchange messages in both directions over a single persistent connection, which makes real-time applications possible without constant polling. Socket.IO builds on this foundation with named events, acknowledgements, rooms, namespaces, middleware, and automatic reconnection, so that developers can focus on the logic of the application instead of the details of the connection. Mastering these concepts is essential to build chats, live dashboards, notifications, and collaborative tools with Node.js.